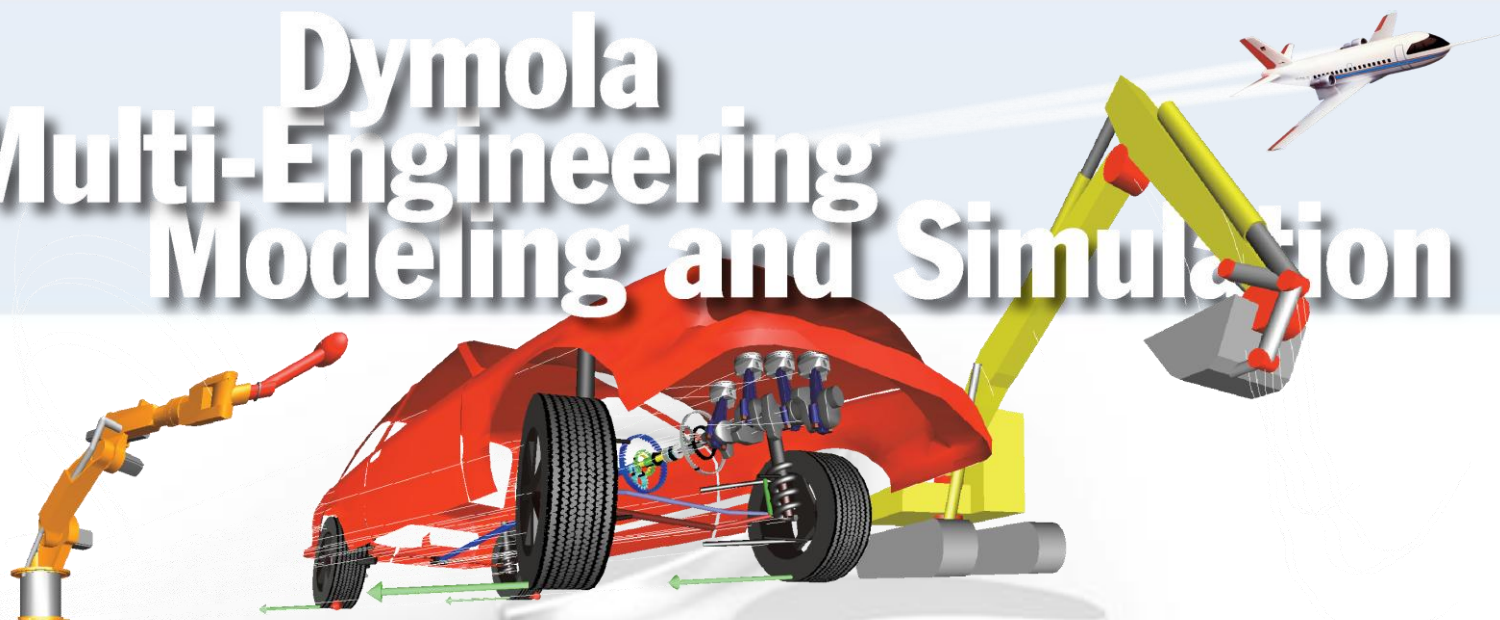


Dymola Multi-Engineering Modeling and Simulation



Dymola 多学科建模与仿真

达索系统解决方案销售 包大石

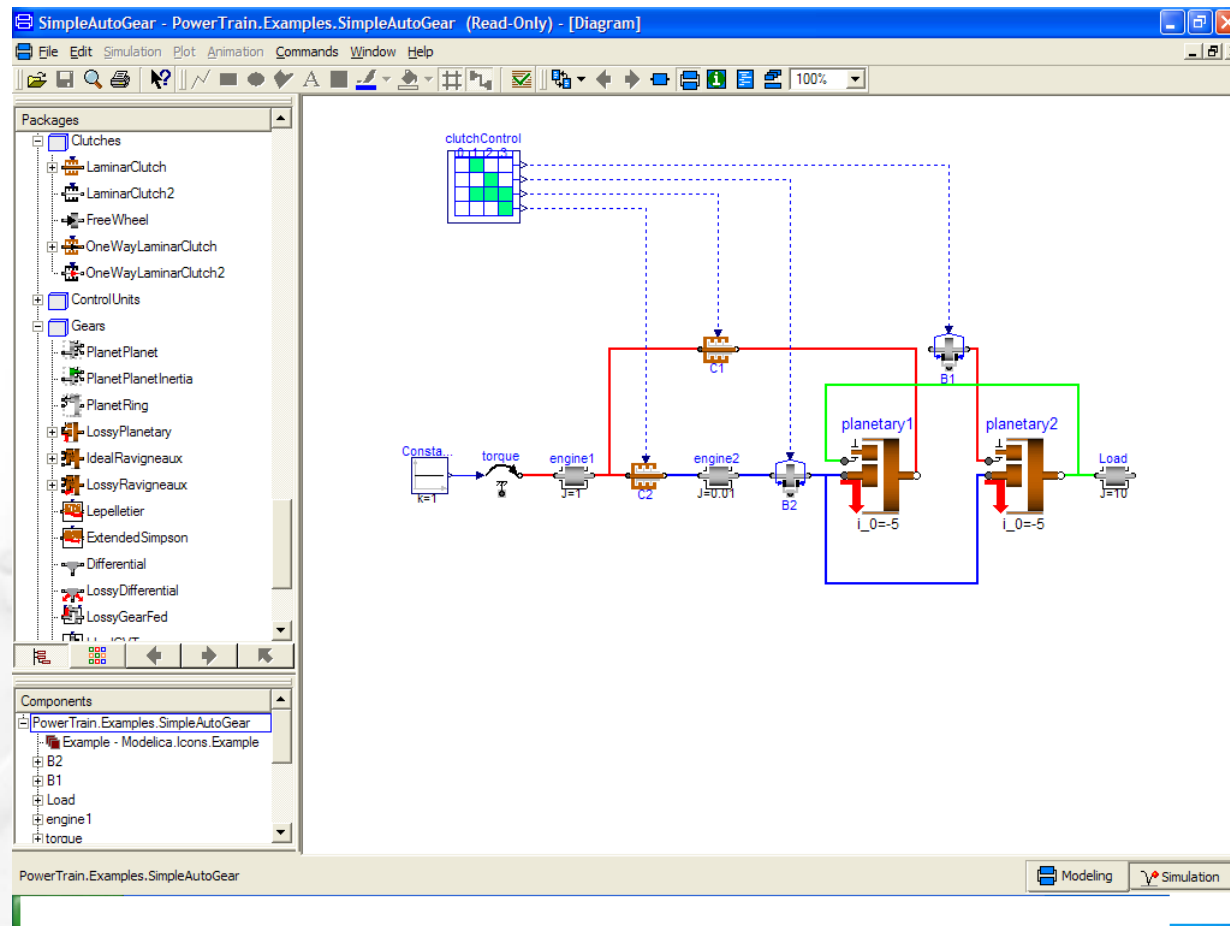
Dymola 发展历史



- 由瑞典**Dynasim AB**公司开发
 - 位于瑞典隆德市
 - 现在是达索系统的全资子公司
 - 于2006年收购
- 简史
 - 最初的面向对象建模和自动方程处理的构想是是Hilding Elmqvist博士在上世纪70年代提出
 - 随着Intel奔腾处理器的诞生，构想成为现实，于1992年创建Dynasim公司，使用Dymola语言
 - 1996年，由Dynasim公司倡导开发了一种新的建模语言来统一仿真建模语言的工作
 - 1999年，成立了Modelica语言协会，正式发布Modelica语言

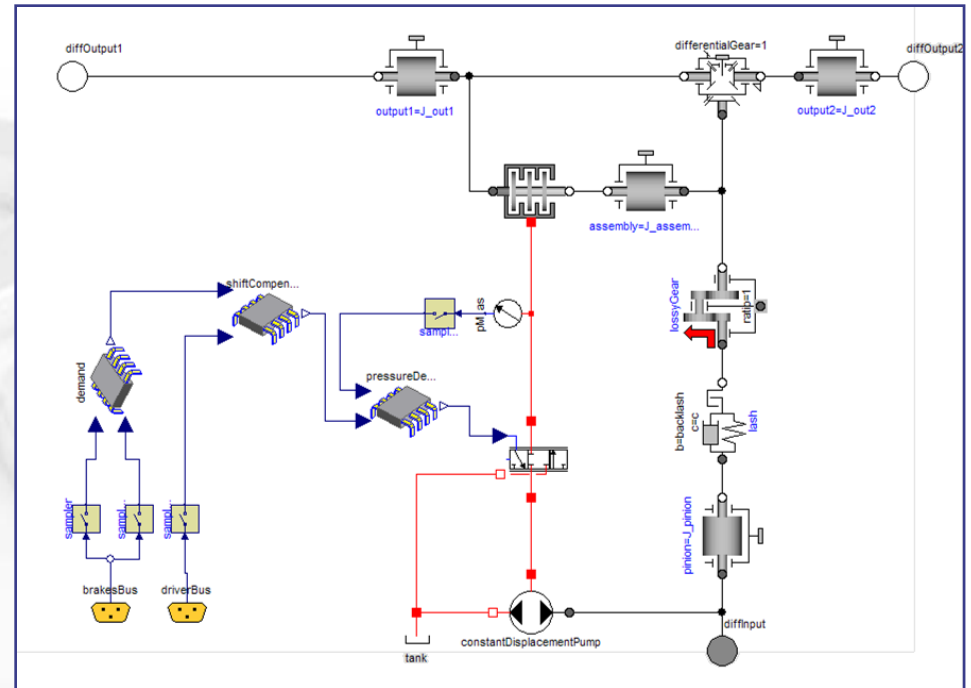
Dymola

- 一个面向对象的集成了多物理学科的系统建模和仿真工具



多学科建模

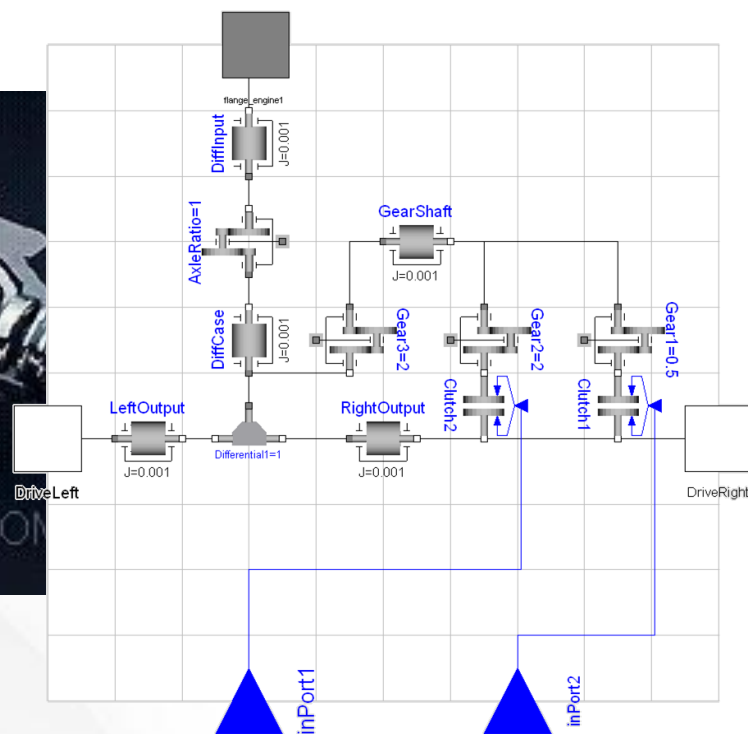
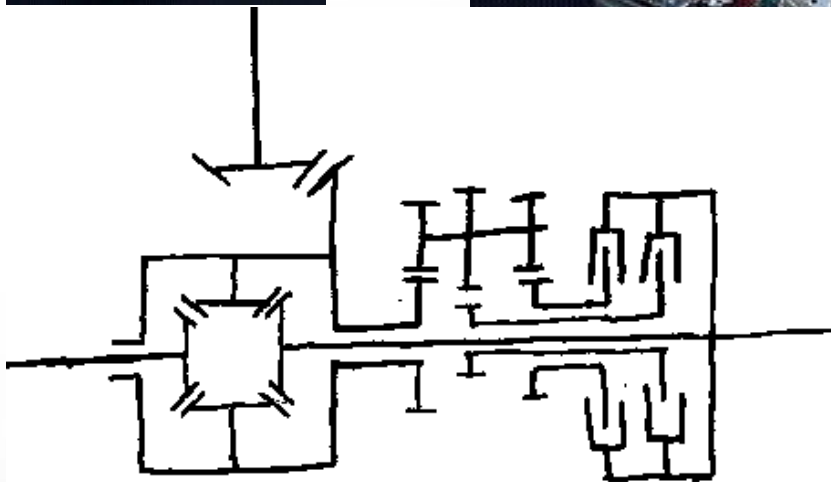
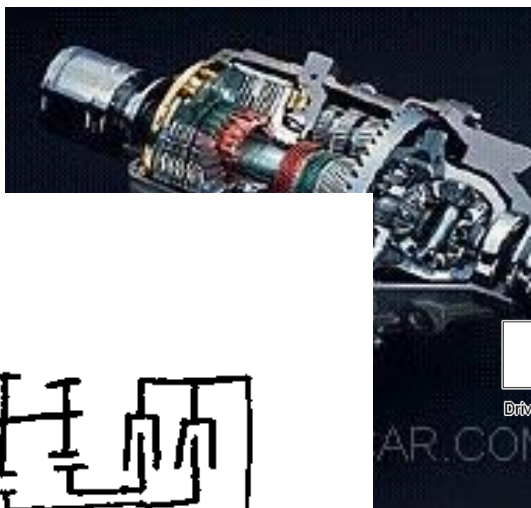
- **Dymola** 支持多学科工程领域的建模，并提供多学科间的适当的连接与交互
- 这些对象代表了系统中的物理零部件
 - 油压（红色）
 - 控制（蓝色）
 - 机械
 - 一维旋转信号（灰色）



一维旋转仿真

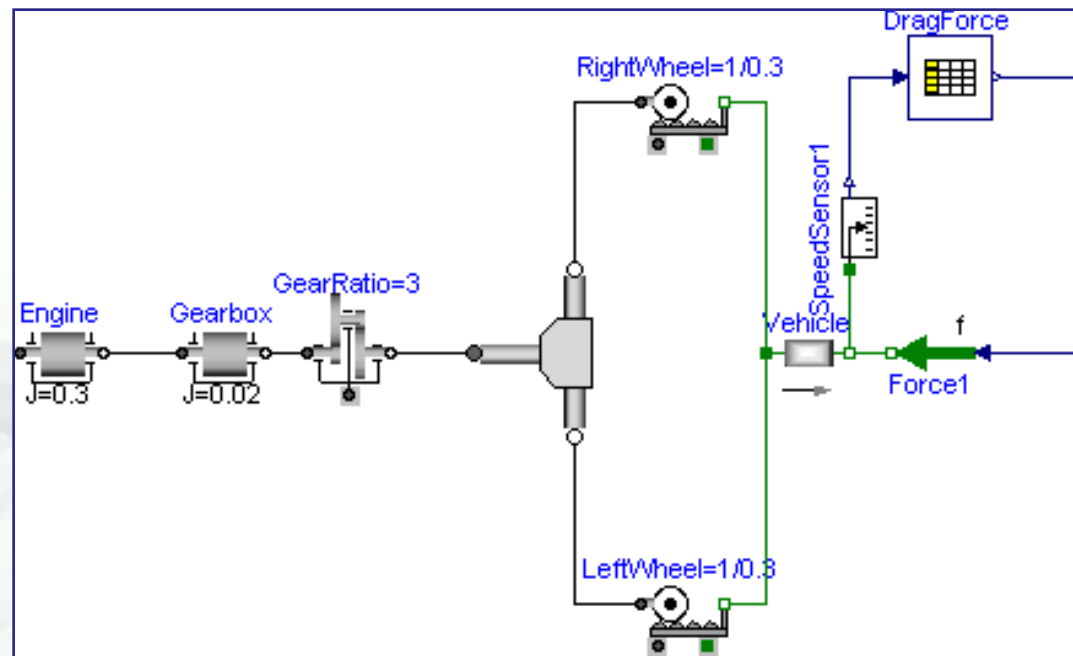
模型建立

- 模型的建立是来源于内嵌的对象（部件）库的调用和连接，形成类似于原理图（**Schematic Drawing**）的表达方式



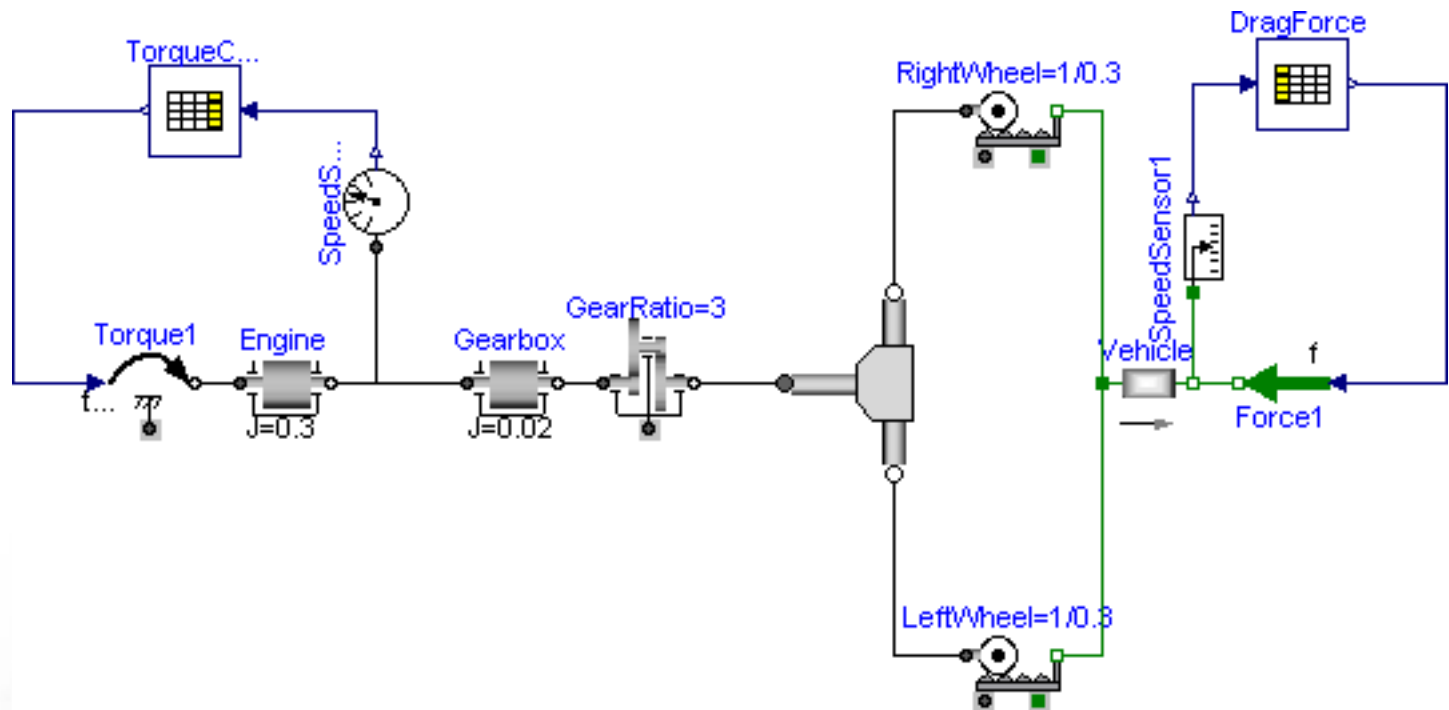
模型重用

- **Dymola**使用的面向对象的建模方式提供更广泛的模型重用
- 一旦模型或系统定义好，它可以被重用到许多不同的分析条件



简单底盘概念模型

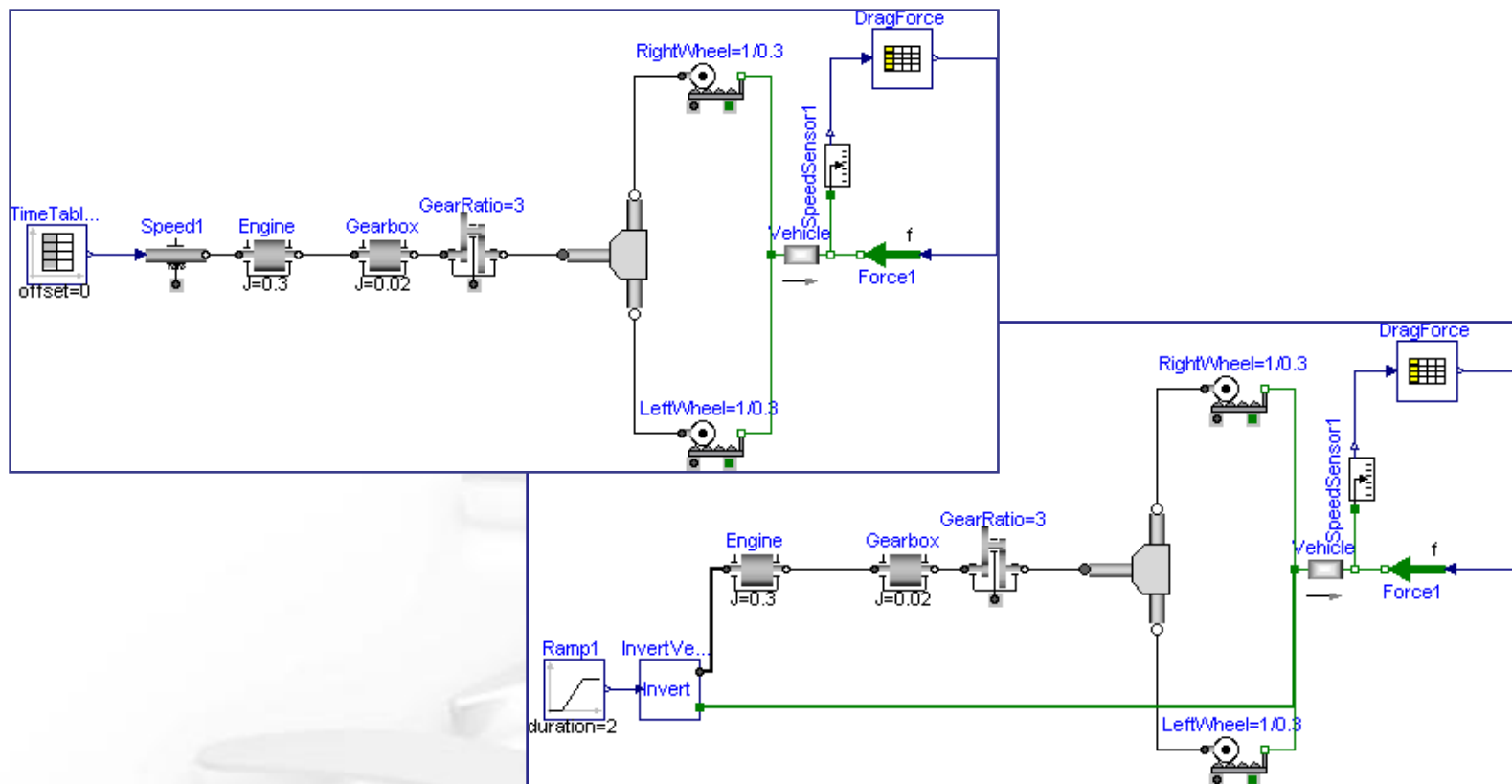
模型重用



- 前向动态模型 **Forward dynamic model**

- 将力矩施加到发动机系统，预测系统的反应，比如加速性

模型重用



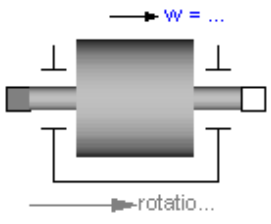
- 逆向动态模型 **Inverse dynamic model**

- 发动机收一个速度-时间曲线驱动，从而计算出发动机需要的力矩

HOW DOES THIS WORK?

面向对象的模型定义方法

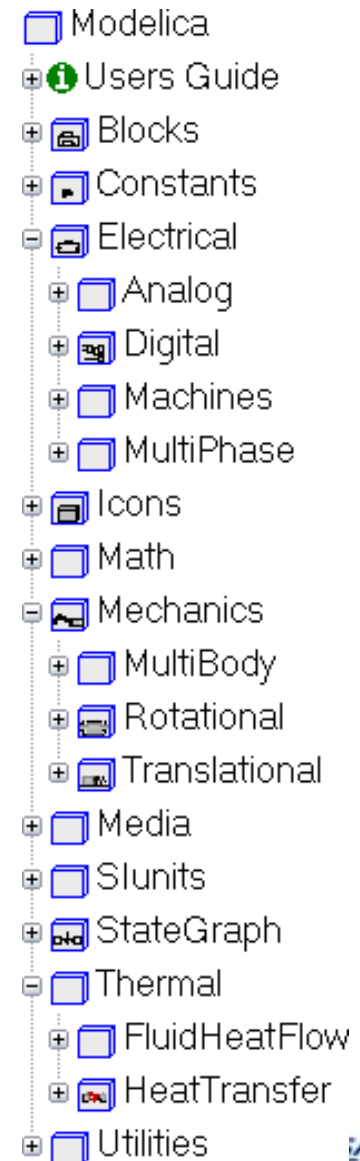
- 模型使用**Modelica** 建模语言构建
 - 一个通用的建模语言
 - 方便，面向对象的建模方式，支持复杂的多学科仿真建模
- **Dymola** 允许访问模型后端的**Modelica** 代码



```
model Inertia
  extends Interfaces.Rigid;
  parameter SI.Inertia J=1 "Moment of Inertia";
  SI.AngularVelocity w "Angular velocity";
  SI.AngularAcceleration a "Angular acceleration";
equation
  w = der(phi);
  a = der(w);
  flange_a.tau + flange_b.tau = J * a;
end Inertia;
```

- 免费开放的建模语言
- 由**Modelica** 协会开发并维护
 - 一个建于1996年独立的非盈利机构
 - 有超过50 个来自于学院和企业的会员
 - 任何人都可以参与
- **Modelica** 标准库包含多学科基本的模型
 - Dymola作为标准的配置提供

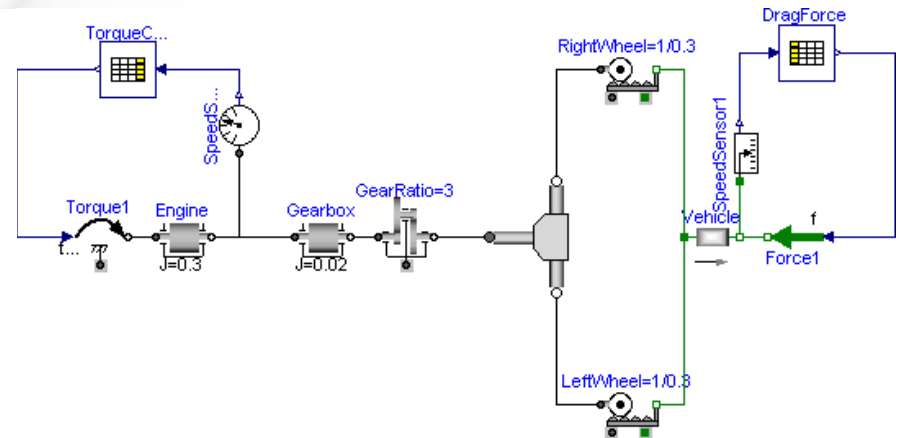
<http://www.modelica.org>



Dymola 和 Modelica

- Modelica 用来描述要仿真的系统
- Dymola 提供
 - 图形化的建模环境，仿真引擎和分析工具
 - 一个编译器，将Modelica 代码翻译成有效的仿真代码

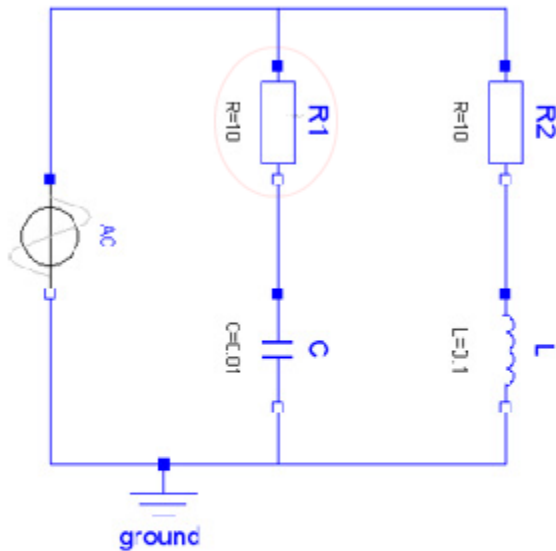
```
model DriveByTorque
  extends SimplePowertrain;
  B;
  Modelica.Mechanics.Rotational.Torque Torque1 B;
  Modelica.Mechanics.Rotational.Inertia Engine(J=0.3, w(start=40)) B;
  Modelica.Mechanics.Rotational.Sensors.SpeedSensor SpeedSensor2 B;
  Modelica.Blocks.Tables.CombiTable1D TorqueCurve(table=[0, 0; 70, 100; 150,
    150; 250, 180; 400, 170; 600, 120; 650, 0; 660, 0]) B;
equation
  connect(Engine.flange_b, Gearbox.flange_a) B;
  connect(Torque1.flange_b, Engine.flange_a) B;
  connect(Engine.flange_b, SpeedSensor2.flange_a) B;
  connect(SpeedSensor2.w, TorqueCurve.u[1]) B;
  connect(TorqueCurve.y[1], Torque1.tau) B;
end DriveByTorque;
```



符号处理

- 模型方程会自动编译成需要的解决方案用于仿真
- 高级的计算技术用来减少出现问题的规模，而不用减低模型的详细程度

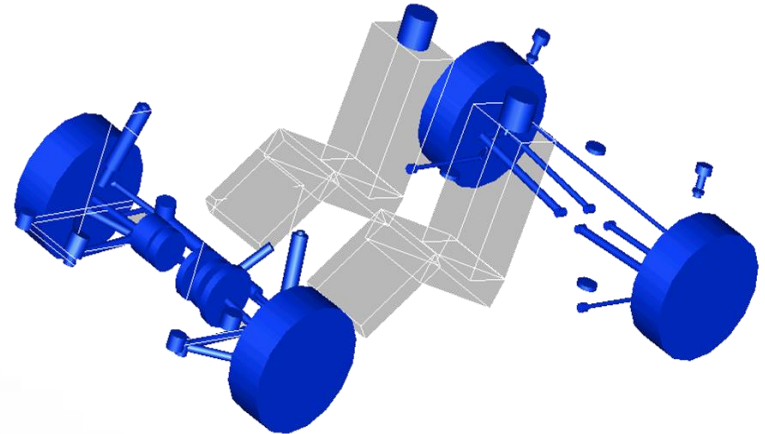
DAE:



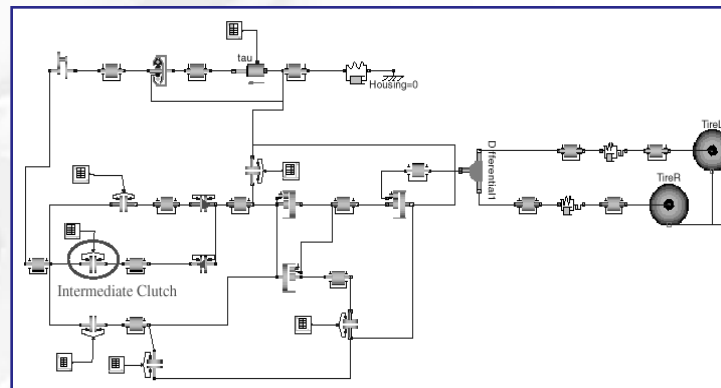
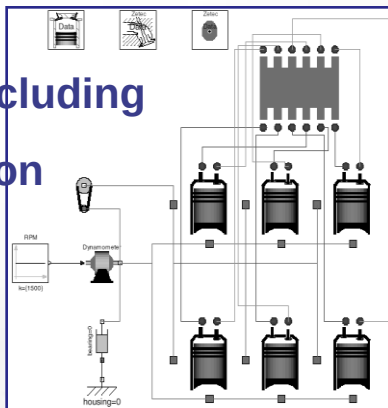
What does this mean in practice

- **Example: Vehicle model with detailed engine, transmission and suspension system**
 - ~ 250,000 equations before reduction
 - ~ 25,000 equations after reduction

Source: Paper presented by Mike Tiller, Ford Motor Company at the Modelica Workshop, 2000

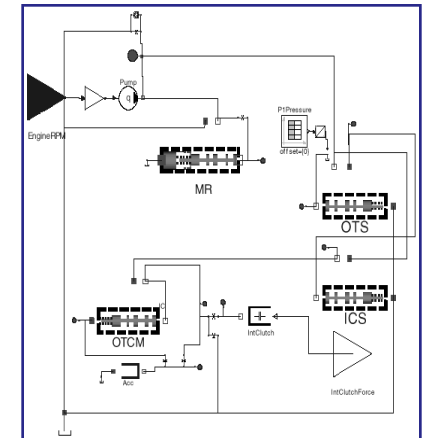


Engine including combustion



Automatic gear box

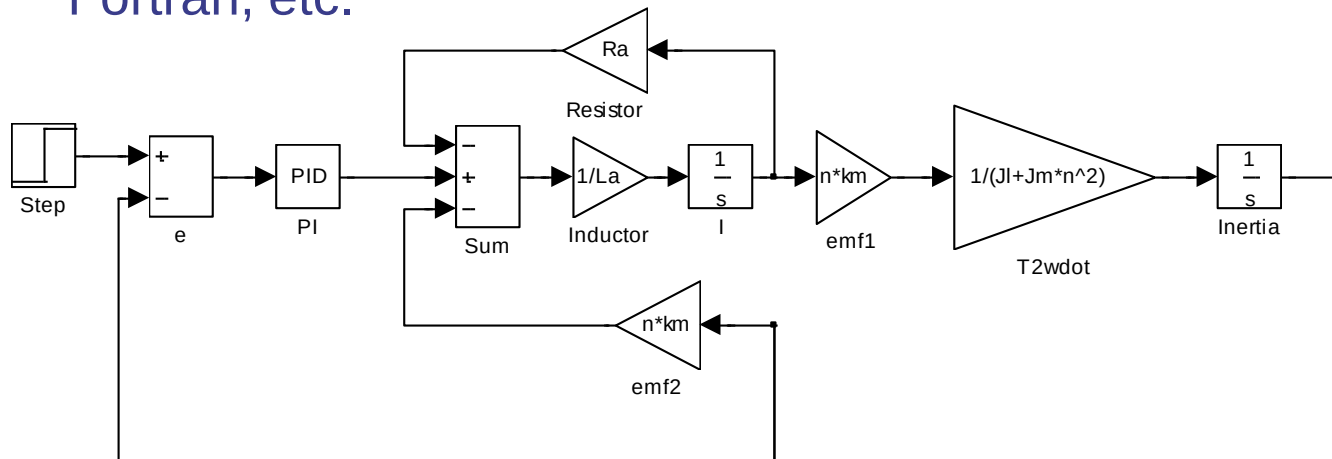
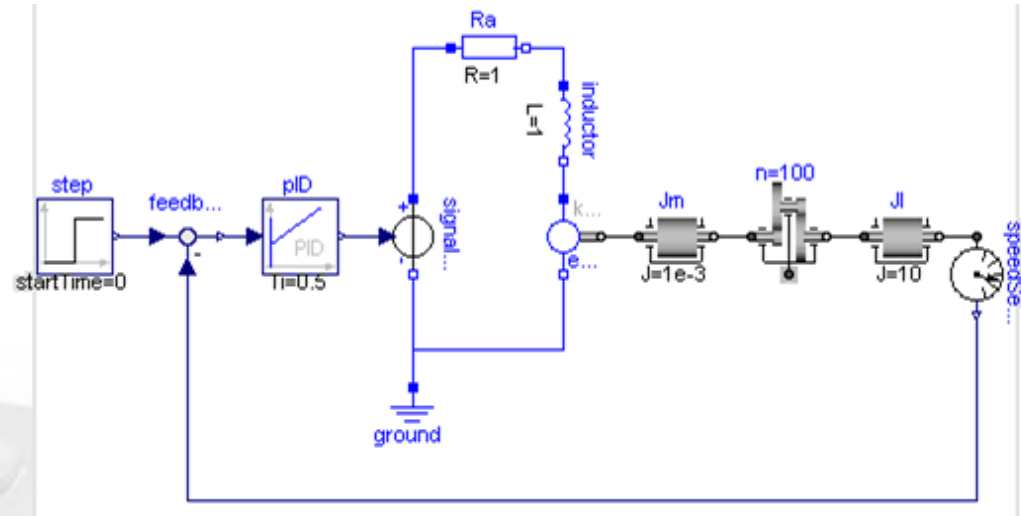
Hydraulics



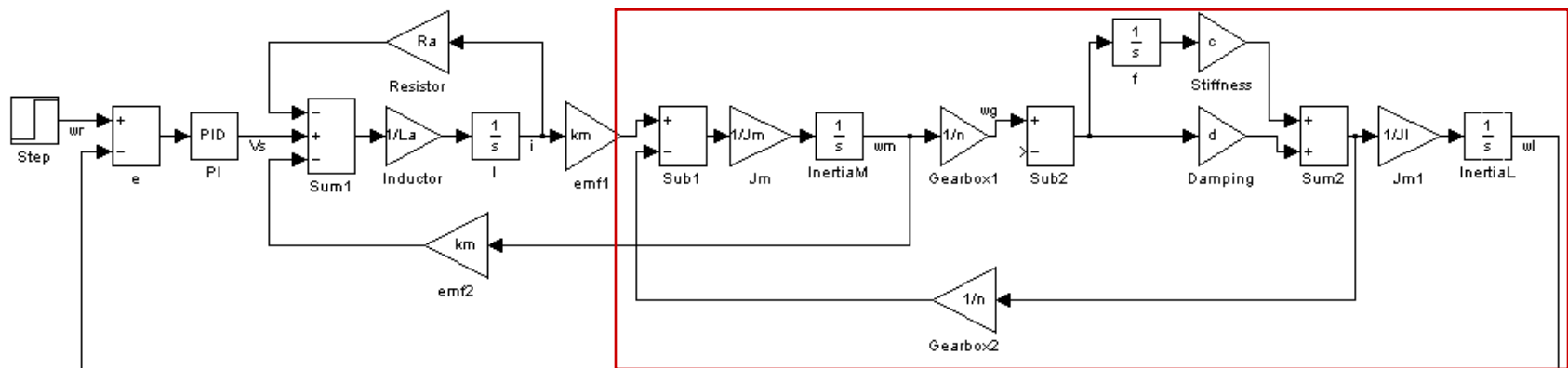
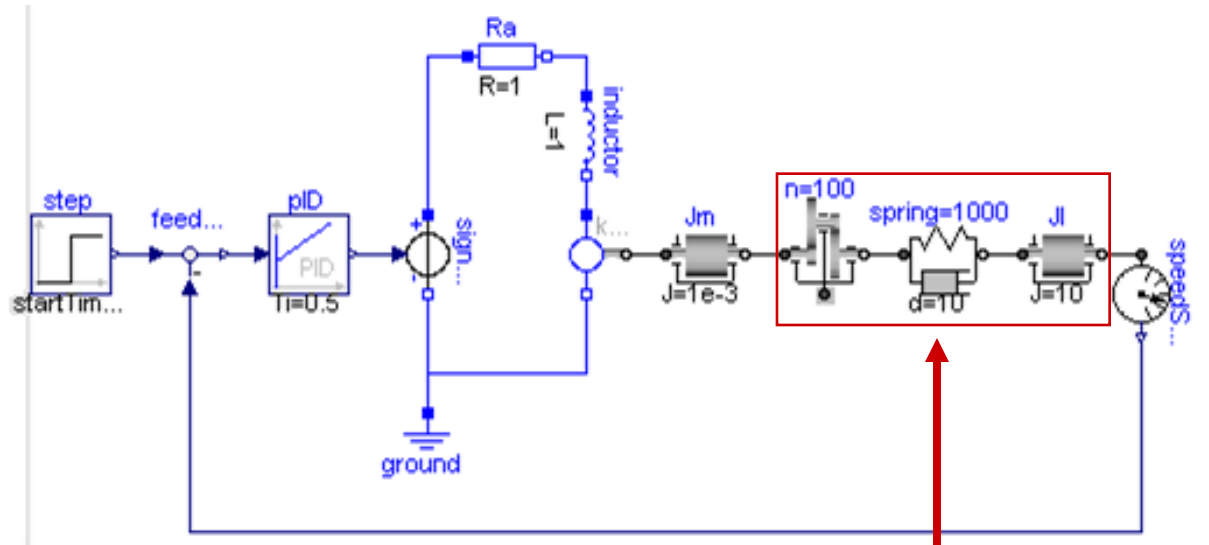
COMPARE DYMOLA AND THE CONVENTIONAL BLOCK DIAGRAM APPROACH TO MODELLING

Component Orientated Modelling

- Simple model to look at the control system behaviour
- Conventional modelling approach
 - 🔧 Block diagram based tools
 - 🔧 Programming in C, Fortran, etc.

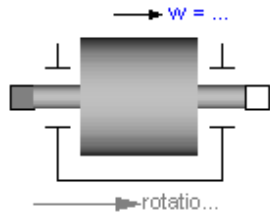


Adding compliance to the system



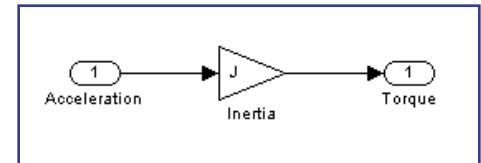
Maintaining physical models

- Dymola requires just one model for a rigid shaft
- Simulink requires four to cover every possible causality

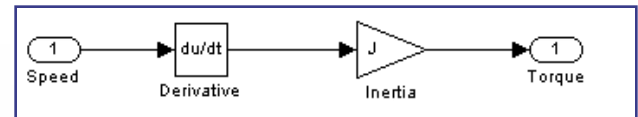


```
model Inertia
  extends Interfaces.Rigid;
  parameter SI.Inertia J=1;
  SI.AngularVelocity w;
  SI.AngularAcceleration a;
equation
  w = der(phi);
  a = der(w);
  flange_a.tau + flange_b.tau = J * a;
end Inertia;
```

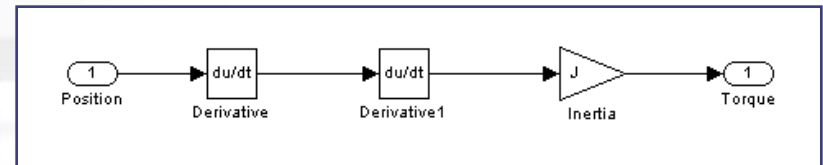
1



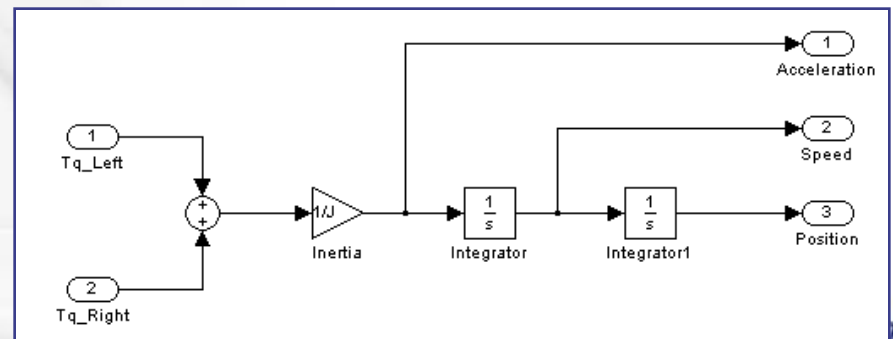
2



3



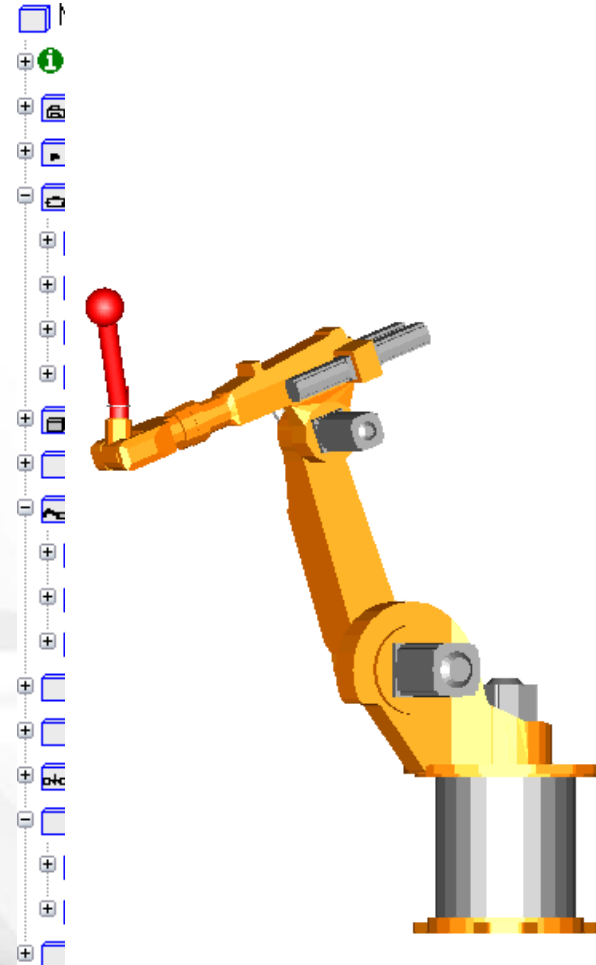
4



MODEL LIBRARIES AND FEATURES

Model Libraries

- **Modelica Standard Library**
 - Wide range of components across many engineering domains
 - Freely available, complete access to source code
- **Powertrain**
- **Smart Electric Drives**
- **Vehicle Dynamics**
- **Air-conditioning**
- **Hydraulics**
- **Pneumatics**
- **Flexible Bodies**



Dymola Features

- **Standard Features**

- Scripting
- Animation
- Java Interface
- Model experimentation
- Real-time simulation on windows

- **Optional Features**

- Simulink interface
- Real-time simulation on HIL platforms
- Model calibration
- Design optimisation
- Model management
- Runtime licenses

Summary

- Dymola is a multi-domain modelling and simulation tool
- Component orientated modelling approach
- Comprehensive suite of model libraries
- Flexibility to add your own models using the Modelica modelling language
- Simulink Interface
- Model Calibration and Optimisation Features
- Real-time capable

